



**UNIVERSITI SAINS MALAYSIA**

School of Electrical and Electronics Engineering  
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

**EEM 441**

Control & Instrumentation Laboratory

## **Experiment 9**

PROJECT CREATION IN SCADA (IOT)

Date : \_\_\_\_\_

Group No. : \_\_\_\_\_

Group Members : \_\_\_\_\_ Matrix # \_\_\_\_\_

: \_\_\_\_\_ Matrix # \_\_\_\_\_

: \_\_\_\_\_ Matrix # \_\_\_\_\_

: \_\_\_\_\_ Matrix # \_\_\_\_\_

## Experiment 9

# IIoT Monitoring & Alarm Visualization for a PLC-Controlled Machine

### Objectives

---

1. To understand the role of each layer in an IIoT monitoring pipeline: PLC, SCADA, database, and dashboard.
2. To configure communication between a Siemens S7-1200 PLC and Advantech WebAccess/SCADA.
3. To bridge SCADA tag data to a PostgreSQL database via ODBC, so that history is available outside WebAccess.
4. To design a web-based IIoT dashboard displaying real-time sensor status and visual alarms, using the WISE-IoTSuite/Dashboard application.
5. To demonstrate a complete monitoring pipeline, from a physical sensor to a live dashboard panel.

### Equipment / Software Required

---

- MTA SkillLab i-Factory kit: conveyor sorting machine with Siemens SIMATIC S7-1200 CPU 1212C AC/DC/Rly, fitted with vibration, laser-distance, temperature/humidity, and part-counting sensors
- PC/laptop with Advantech WebAccess/SCADA V9.x installed (Project Node and SCADA Node)
- PostgreSQL, with the psqldb (x86) ODBC driver and pgAdmin4 installed
- ODBC Data Source Administrator (32-bit) — WebAccess only supports 32-bit ODBC connections
- Ethernet cable and a configured local network connection between the PC and the PLC

## Introduction

Industrial IoT (IIoT) monitoring bridges field-level operational technology (OT) – sensors, actuators, and the PLC that controls them – with IT-side visualization and analytics. This experiment builds that bridge one layer at a time, using the same i-Factory conveyor rig and WebAccess/SCADA software introduced in Experiment 5.

**Communication protocols used at each layer** (Table 9.1): Modbus links the sensors to the PLC; TCP/IP (or OPC UA) links the PLC to WebAccess/SCADA; and ODBC links WebAccess/SCADA to the PostgreSQL database that the Dashboard ultimately reads from.

The full pipeline is shown in Figure 9.1. Each stage depends on the one before it: a tag with no PLC communication shows no data; a tag with communication but no **Log Data** setting never reaches the database; and a database with no rows has nothing for the Dashboard to display. Troubleshooting this experiment usually means working backward through this chain to find where data stops flowing.

Table 9.1: Communication protocols in the IIoT pipeline.

| # | Protocol | Role                                                               |
|---|----------|--------------------------------------------------------------------|
| 1 | Modbus   | Communication between sensor and PLC                               |
| 2 | TCP/IP   | Communication between PLC and WebAccess (SCADA)                    |
| 3 | ODBC     | Communication between SCADA and the database                       |
| 4 | OPC UA   | Alternative standard communication between PLC/equipment and SCADA |



Figure 9.1: IIoT monitoring pipeline for this experiment.

## Sensor Assignment

Each lab group is assigned one primary sensor/parameter to monitor for this experiment. Working with a distinct analog or discrete signal per group ensures each team independently validates its own signal path (PLC addressing, tag configuration, alarm thresholds, and dashboard binding) rather than inheriting settings tuned for a different sensor type.

Refer to Table 9.2 for your group's assigned sensor, its PLC DB address, and the suggested tag prefix to use in WebAccess/SCADA.

Table 9.2: Sensor assignment by group.

| Group | Sensor / Parameter             | PLC DB address | Suggested tag prefix |
|-------|--------------------------------|----------------|----------------------|
| G1    | Temperature                    | DB3, DBD92     | G1_Temp              |
| G2    | Relative humidity              | DB3, DBD88     | G2_Humid             |
| G3    | Vibration — Z-axis RMS         | DB3, DBD76     | G3_VibZ              |
| G4    | Vibration — X-axis RMS         | DB3, DBD72     | G4_VibX              |
| G5    | Vibration sensor temperature   | DB3, DBD80     | G5_VibT              |
| G6    | Laser distance (analog)        | DB3, DBD84     | G6_Dist              |
| G7    | Good part count                | DB3, DBD12     | G7_GoodP             |
| G8    | Defect part count              | DB3, DBD16     | G8_DefP              |
| G9    | Machine OEE availability index | DB3, DBD24     | G9_Avail             |
| G10   | Conveyor motor speed           | DB3, DBD32     | G10_MtrSpd           |

Every group also shares two common digital signals, regardless of assigned sensor: **Start** (DB1, DBX0.1), **Stop** (DB1, DBX0.3), and **Emergency Stop** (DB1, DBX0.0).

Include at least one of these as a discrete tag alongside your assigned analog sensor, so your dashboard has both a trend and a status indicator to display.

## Procedure

### Activity 1 — Create the SCADA project and connect to the PLC

*Objective: create a WebAccess/SCADA project and bring your assigned sensor in as a live tag.*

1. Open a web browser and navigate to 127.0.0.1 (or localhost) to reach the WebAccess/SCADA landing page. Select **Project Management** from the **Platform** dropdown and log in (default account admin, no password).

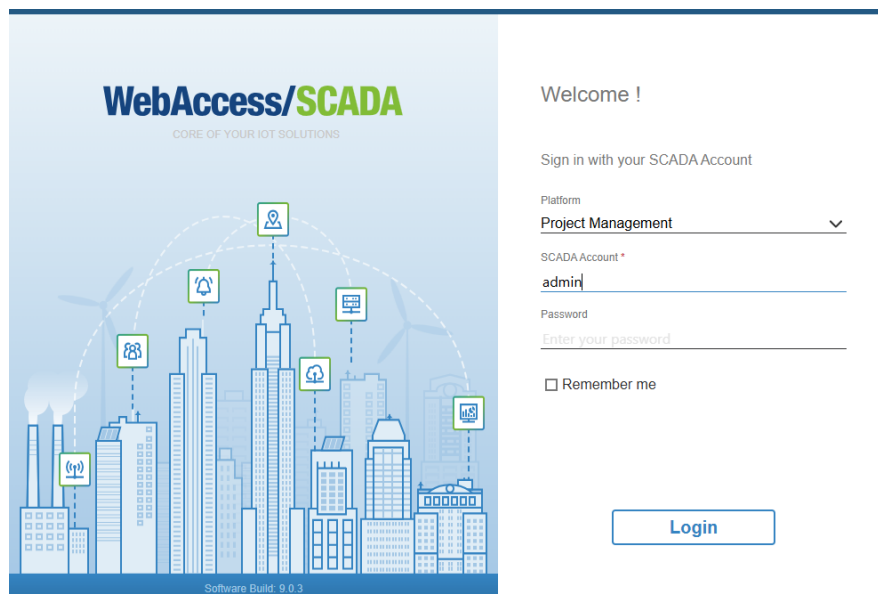


Figure 9.2: WebAccess/SCADA sign-in, with Project Management selected.

2. Create a new project: enter a **Project Name** and **Description**, set the **Project Node IP Address** (127.0.0.1 for local), and keep the default **Primary TCP Port** (4592) and **HTTP Port** (80) unless instructed otherwise.

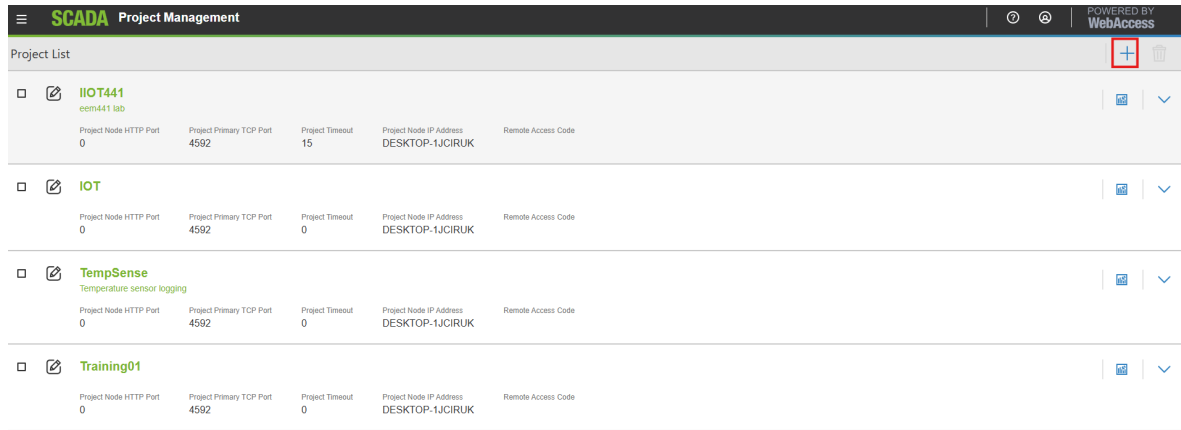


Figure 9.3: Create New Project form.

3. Create a **SCADA Node**: select your Project Node, click **+**, choose **New Node**, and fill in the Primary Node Name, TCP Ports, Node Timeout, and SCADA Node IP Address.



Figure 9.4: Create New SCADA Node form.

4. Add a **Communication Port**: choose **TCP/IP** as the interface, and configure the Scan Time (e.g. 1000 ms) and Auto-Recover Time.

**Create New Comport**  
\* Required

Interface Name  
TCP/IP

Comport Number \*  
1

Description  
S7-1200 OEEE Monitoring TCP/IP

Scan Time \*  
1000  
4/6 ☒ MilliSecond ☐ Second ☐ Minute ☐ Hour

Timeout (Unit:MilliSecond) \*  
3000  
4/12

Retry Count \*  
3  
1/12

Auto Recover Time (Unit:Seconds) \*  
5

CANCEL SUBMIT

Figure 9.5: Communication port (Comport) settings.

5. Add a **Device** on that Comport: set Device Type to SiemS7, enter the PLC's IP address and port (102), and set the TSAP in hex for an S7-1200 (01.01).

**Create New Device**  
\* Required

Device Name \*  
Siemens PLC

Description  
SIEMENS S7-1200

Unit Number \*  
0

Device Type  
SiemS7

Manually switch Primary/Backup. \*  
0

TSAP in Hex :Device ID, RackSlot \*  
03.02

☐ Primary

CANCEL SUBMIT

Figure 9.6: Add Device form (S7-1200).

6. Create an IO Tag for your assigned sensor, using the prefix from Table 9.2 (e.g. G1\_Temp), and a second discrete tag for Start, Stop, or Emergency Stop. Set the Address, Data Type, and Conversion Code to match your PLC DB offset.

**Create New Tag**  
\* Required

**Basic Information**

Tag Type  
DBD (analog)

Tag Name \*  
TemperatureSensor

Description  
DB DWord Data

Scan Type  
Constant Scan

Address \*  
DBD3,96

Conversion Code  
Unsigned Integer

Start Bit \*  
0

Length \*  
32

Signal Reverse  
☐ Yes ☒ No

Scaling factor 1 \*  
0

Scaling factor 2 \*  
0

CANCEL SUBMIT

Figure 9.7: Insert IO Tag form (analog sensor example).

7. Start the SCADA Node's Kernel, open **ViewDAQ**, and confirm both tags are updating with live values that reflect what is actually happening on the machine.

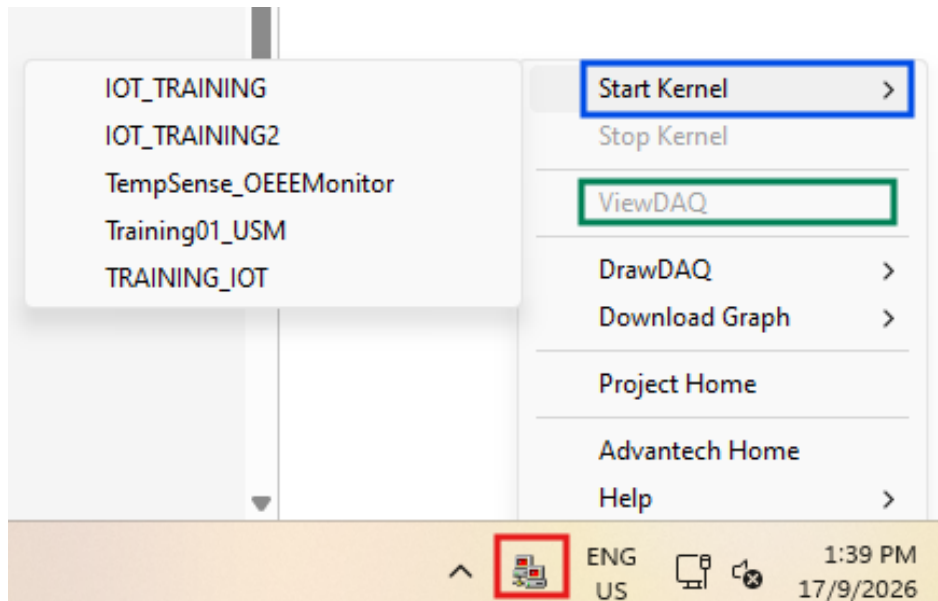


Figure 9.8: Kernel running, viewed in ViewDAQ.

## Activity 2 – Configure an alarm

*Objective: enable alarming on your assigned sensor, so threshold breaches are visible in both SCADA and, later, the dashboard.*

1. Open your analog sensor tag's configuration and enable a non-zero alarm priority (High, High-High, Low, or Low-Low, as appropriate to your sensor).

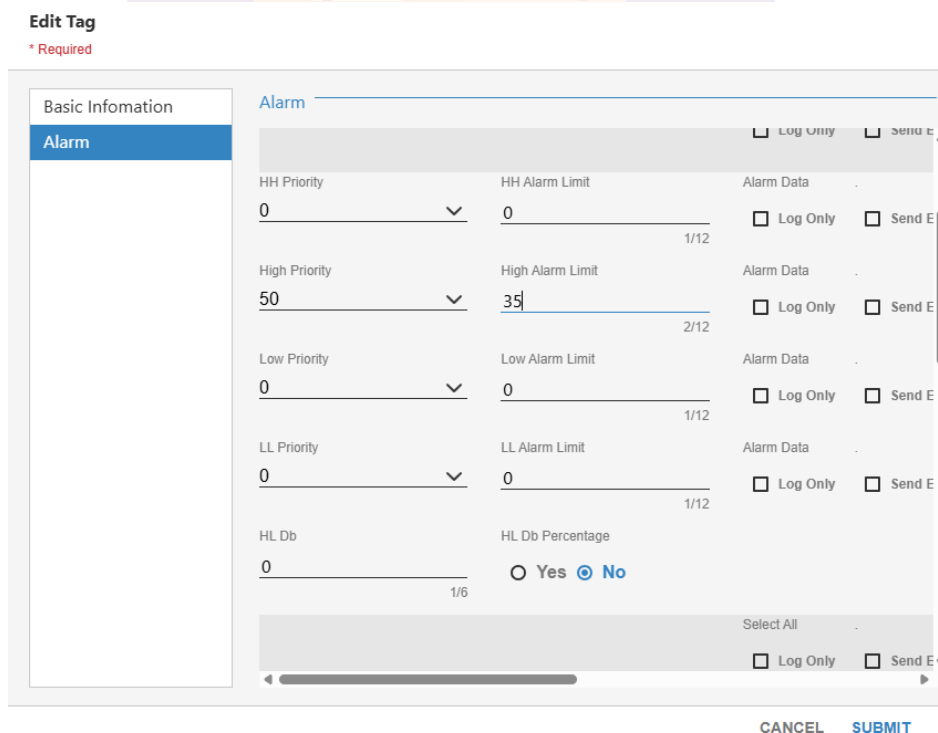


Figure 9.9: Alarm priority configuration.

2. Enable state alarming on your discrete tag (e.g. alarm when Emergency Stop becomes active).

3. In **ViewDAQ**, trigger a manual threshold breach (e.g. briefly block the sensor's path, or press the Emergency Stop) and confirm the alarm appears in the SCADA **Alarm Summary**.

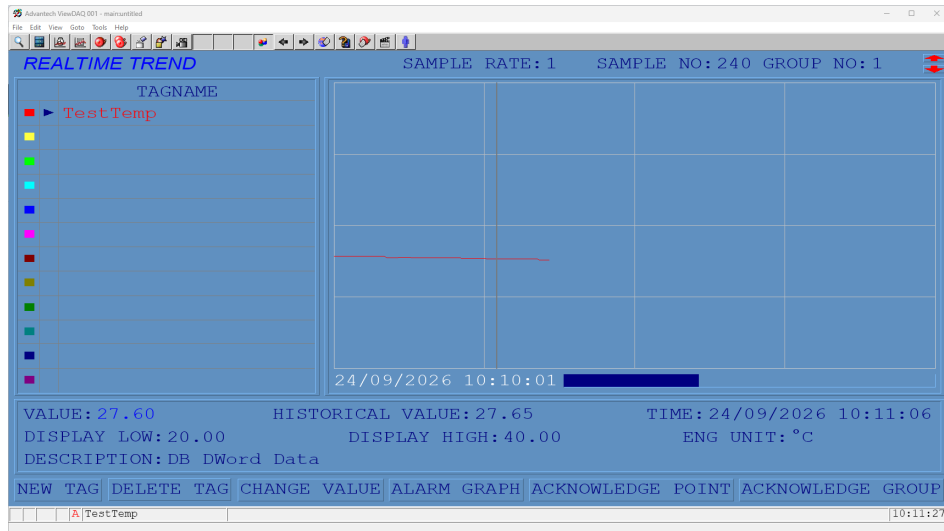


Figure 9.10: Alarm triggered, viewed on the real-time trend.

### Activity 3 – Bridge SCADA to PostgreSQL via ODBC

*Objective: connect WebAccess to a PostgreSQL database, and enable logging so your tag's history is written there.*

1. Confirm with your instructor that PostgreSQL, the `psqlodbc` (x86) driver, and `pgAdmin4` are already installed — this experiment assumes they exist; installing them from scratch is outside the scope of this lab.
2. In `pgAdmin`, create a connection to the PostgreSQL server (Host `127.0.0.1`, Port `5436`, default credentials provided by your instructor), then create a new database for this experiment.

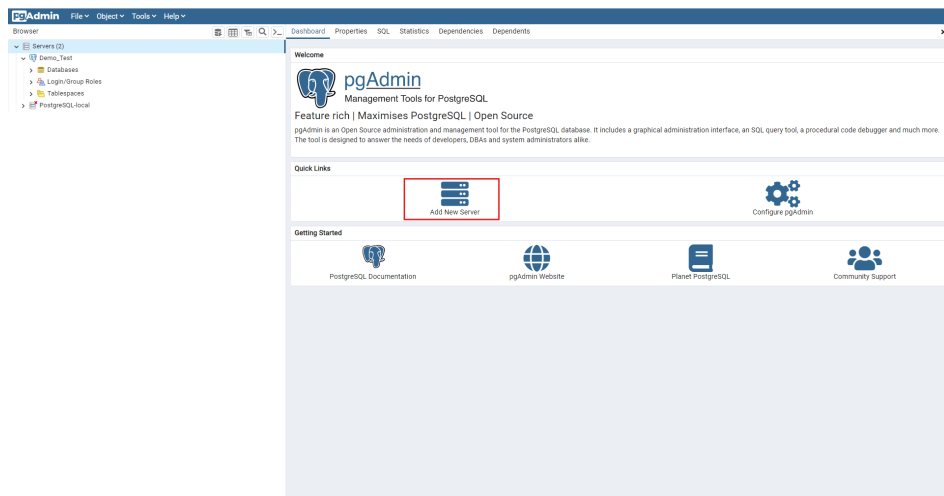


Figure 9.11: pgAdmin: registering the PostgreSQL server.

- Open **ODBC Data Source Administrator (32-bit)**, switch to the **System DSN** tab, click **Add**, and select **PostgreSQL Unicode**. Fill in the Data Source name, Database, Server, Port, Username, and Password to match the database you just created.

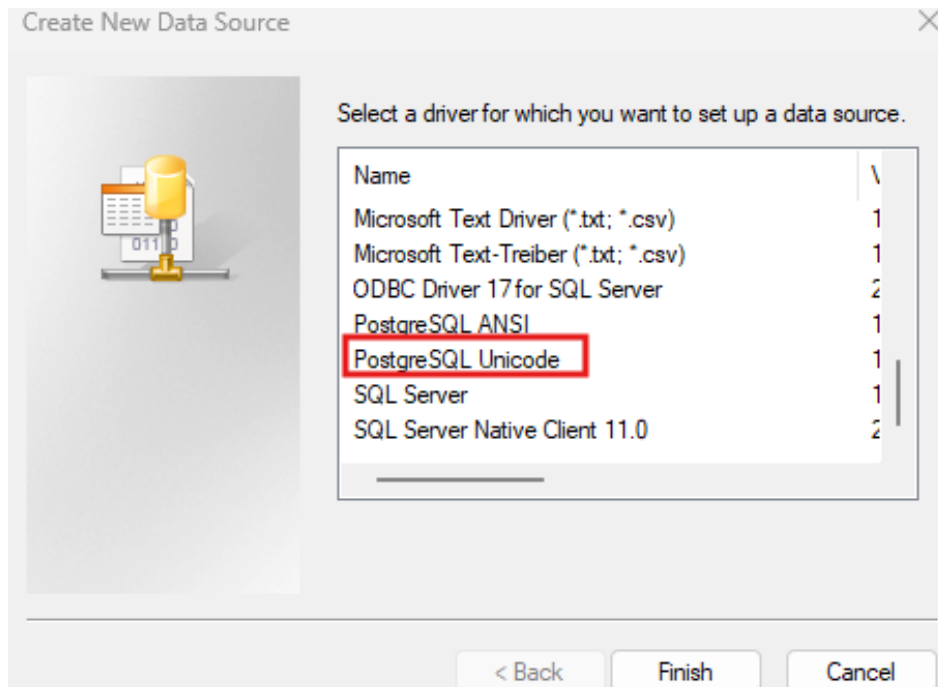


Figure 9.12: ODBC Data Source Administrator (32-bit).

- Back in WebAccess, open the project's **Setting** menu → **ODBC data record source**, and insert a new ODBC entry: a numeric Log Data Source ID, the DSN name from the previous step, and matching login credentials. Test the connection.

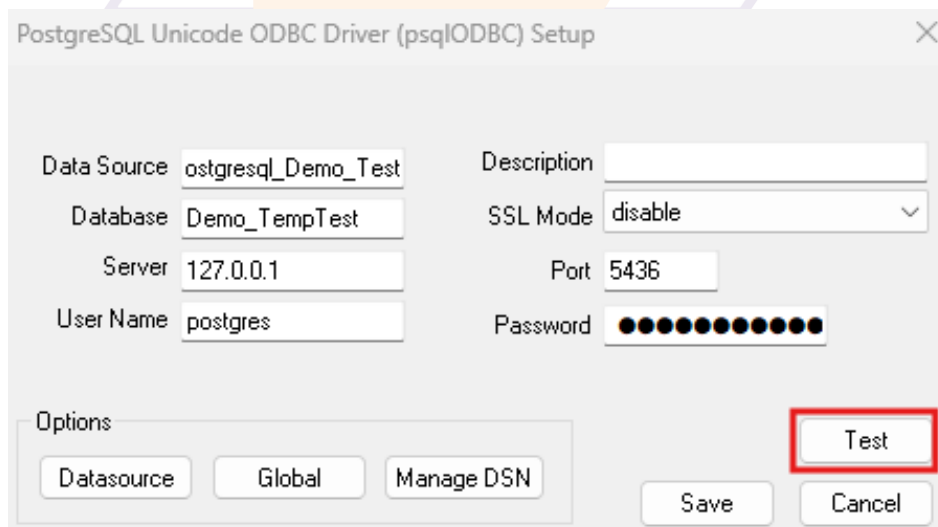


Figure 9.13: WebAccess ODBC data record source.

- On your SCADA Node, enable **Data Log to ODBC (Edit Node → Data Log)**. Then, on each tag you want logged, enable **Log Data** and set the **Data Log Interval** to 0 (record by deadband) to avoid missed samples.

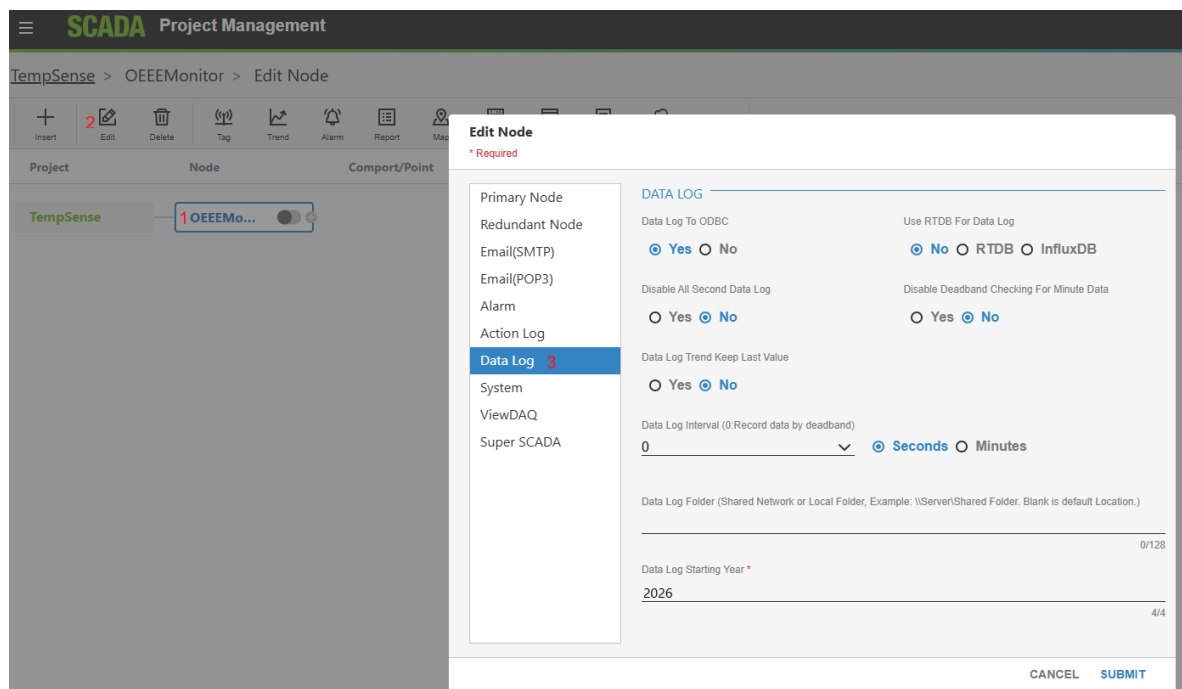


Figure 9.14: SCADA Node Edit Node: Data Log tab.

- Download the project to the SCADA Node, then confirm in pgAdmin that rows are appearing in the corresponding log table (e.g. bwanaLogtable for analog tags).

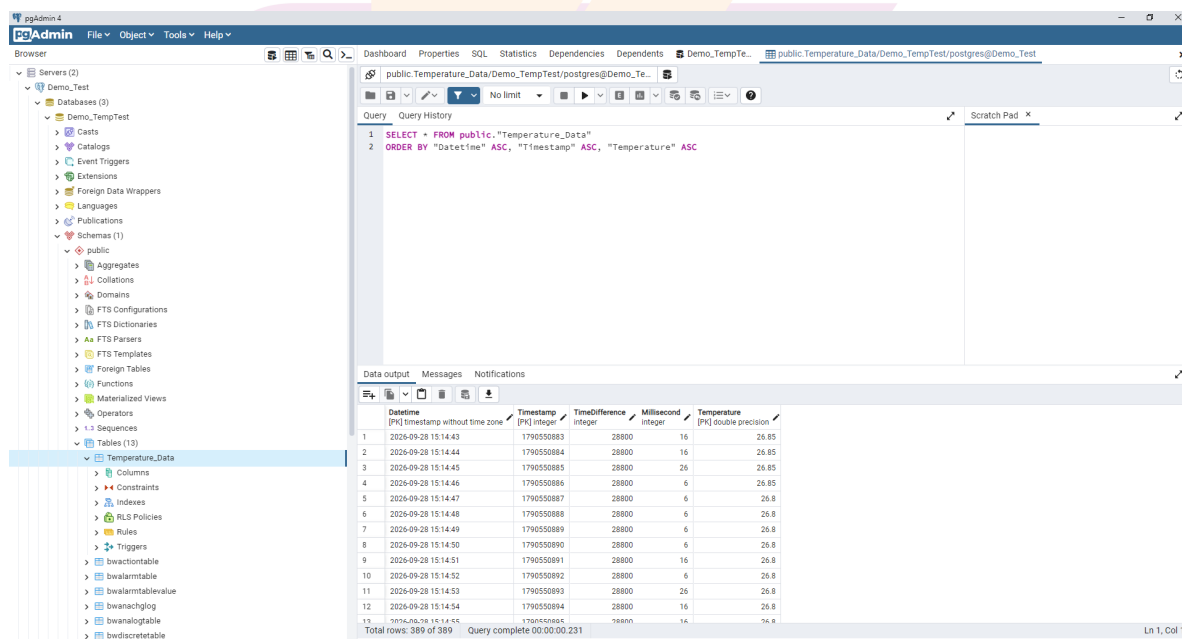


Figure 9.15: pgAdmin: logged rows in the analog log table.

### Activity 4 – Build a dashboard

*Objective: visualize your assigned sensor and its alarm state on a WISE-IoTSuite/Dashboard panel.*

1. From the WebAccess landing page, select **Dashboard** from the **Platform** drop-down instead of Project Management.
2. In the Dashboard app, open **Configuration** → **Data Sources** → **Add data source**, choose **PostgreSQL**, and enter the same connection details used for the ODBC bridge in Activity 3. Click **Save & Test** and confirm a successful connection.

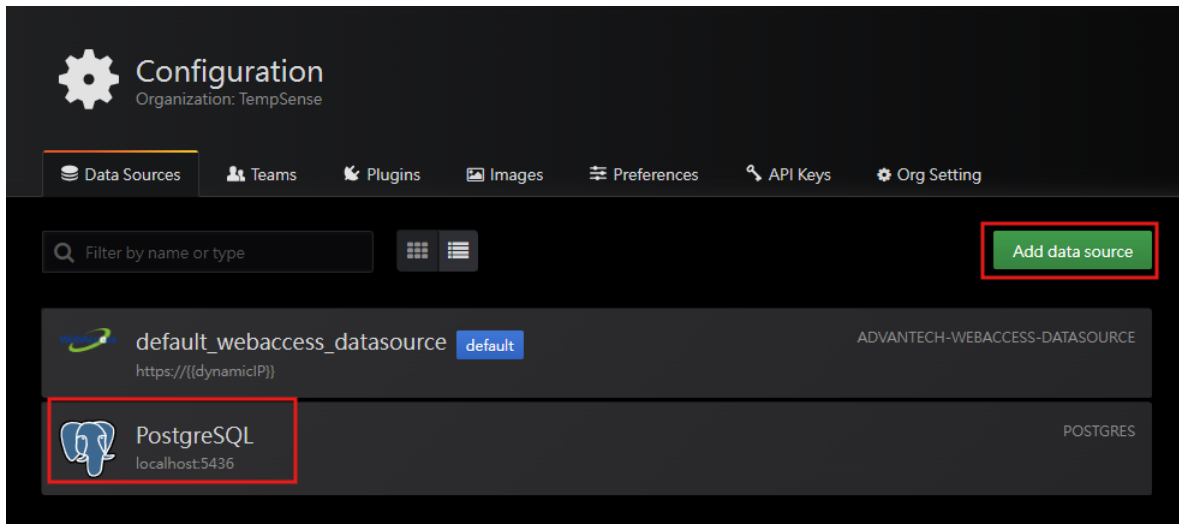


Figure 9.16: Dashboard: Add PostgreSQL data source.

3. Create a **new Dashboard**. Add at least two panels:
  - a **Singlestat** or **Graph** panel bound to your assigned analog sensor tag, and
  - an **Alert-enabled** panel (or an Alert list) showing the current state of your alarm from Activity 2.

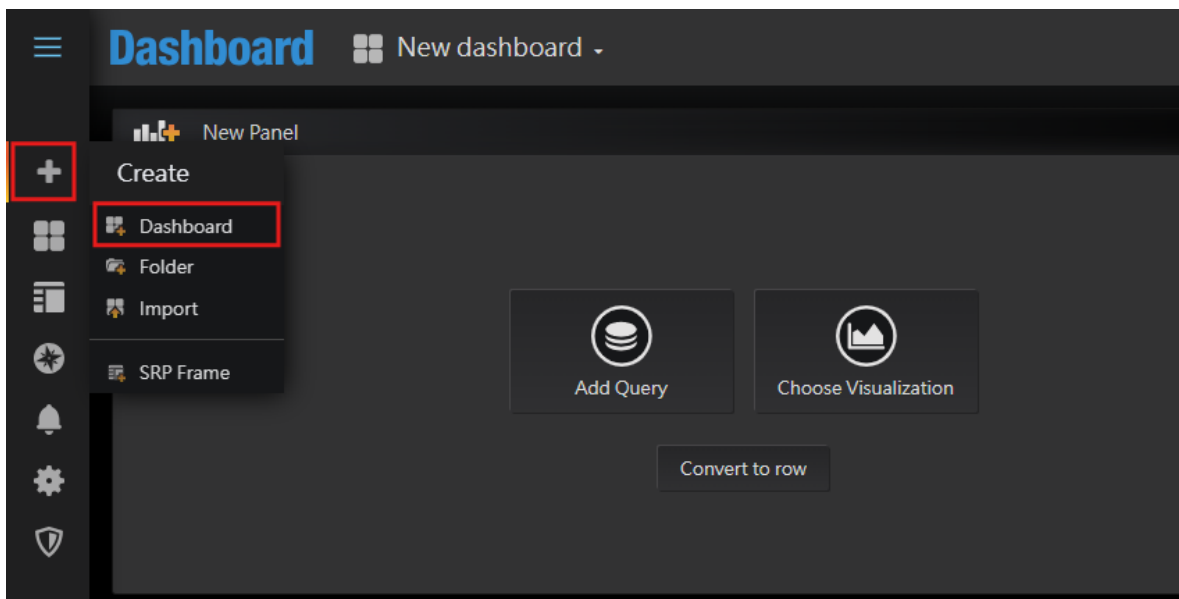


Figure 9.17: Dashboard panel query binding.

4. For each panel, select your PostgreSQL data source and the query/tag that corresponds to your assigned sensor.
5. Label your dashboard clearly with your group number, so it is identifiable in your submitted report.
6. With the machine running, confirm each panel updates live, and that triggering your alarm condition changes the alert panel's state.

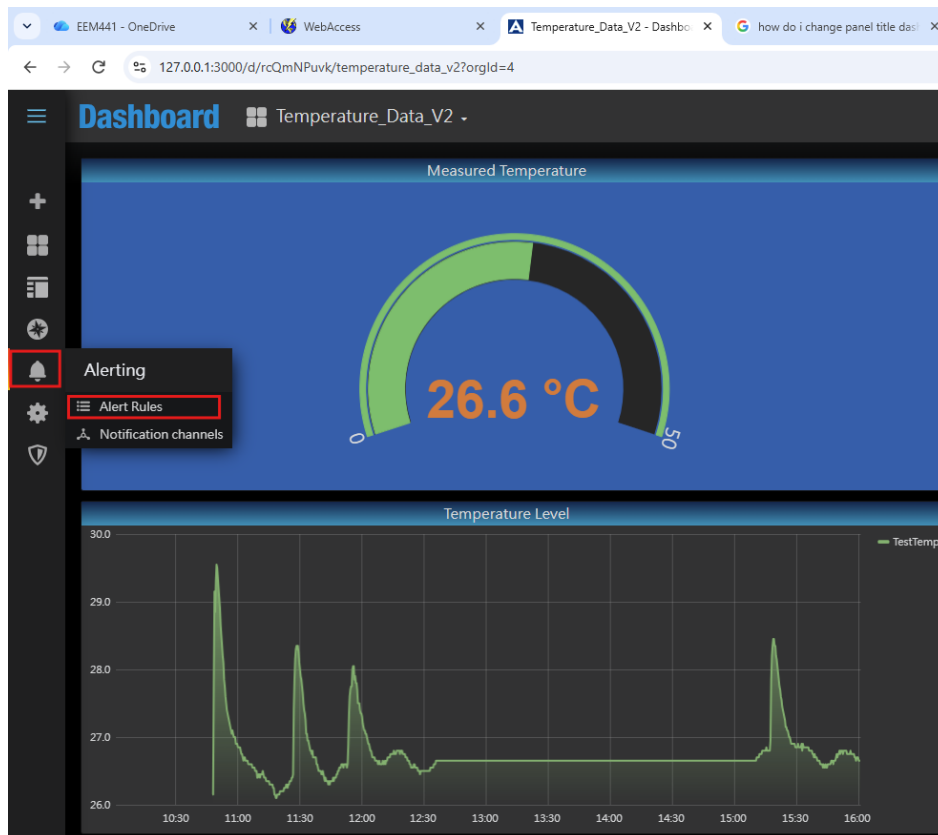


Figure 9.18: Finished dashboard example.

## Report

1. Draw the complete data pipeline for your system (PLC → SCADA → database → dashboard), labelling the protocol used at each stage.
2. State your assigned sensor, its PLC DB address, and the WebAccess tag configuration you used to bring it into SCADA.
3. Describe the alarm you configured, including its threshold and priority, and show evidence (e.g. a screenshot of the Alarm Summary) that it triggers correctly.
4. Describe your ODBC bridge configuration (DSN name, database, and the WebAccess ODBC data record source settings), and explain what would go wrong if **Log Data** were left disabled on your tag.
5. Present a screenshot of your finished dashboard, and explain why you chose the panel types you used for your sensor and alarm.

6. Describe one problem you encountered in this experiment (in any layer of the pipeline) and how you diagnosed and resolved it.

## Deliverables

---

1. **System configuration summary:** your PLC, SCADA, and ODBC settings, including the pipeline diagram and tag mapping from the Report questions.
2. **Dashboard screenshot(s):** your finished dashboard, clearly labelled with your group number.
3. **Short demonstration:** a live walkthrough (or short video) showing your dashboard updating in response to the physical sensor, and the alarm triggering correctly.

## References

---

- MTA SkillLab Sdn Bhd, *i-Factory IIoT Training Manual*, Module 2 (IIoT Hardware & Software), Module 3 (Project Creation in SCADA), and Module 4 (Dashboard Viewing and Analysis).
- Advantech WebAccess/SCADA product documentation.
- PostgreSQL ODBC driver (psqldbc): <https://www.postgresql.org/ftp/odbc/versions/msi/>

## Appendix: Troubleshooting

---

This experiment's pipeline has four layers (Table 9.1), and almost every symptom traces back to exactly one of them. Work backward from where the data visibly stops:

1. If the dashboard panel is blank, check whether the database has rows; if not, check whether SCADA shows a live value; if not, check the PLC connection.
2. If the database has no rows, check whether SCADA is configured to log data; if not, check whether the SCADA Node is running; if not, check the ODBC connection.
3. If SCADA shows no live value, check whether the tag is configured correctly; if not, check whether the Comport and Device are configured correctly; if not, check the PLC connection.